Task 1: Guided Activity, Library item model

1.Code:

```
enum BookStatus {
    case available
    case borrowed(by: String, dueIns: Int)
    case reserved(for: String)

    var label: String {
        switch self {
        case .available:
            return "Available"
        case .borrowed(let name, let days):
            return "Borrowed by \(name), due in \(days) days"
        case .reserved(let name):
            return "Reserved for \(name)"
        }
    }
}
struct Book {
    var title: String
    var status: BookStatus = .available
}

class Member {
    var name: String
    var borrowedBooks: [String]

    init(name: String, borrowedBooks: [String] = []) {
        self.name = name
        self.borrowedBooks = borrowedBooks
```

```
    }
}
var book1 = Book(title: "Swift Programming")
print("Book Status:", book1.status.label)

var book2 = book1
book2.status = .borrowed(by: "Ben", dueIns: 7)

print("Original Book:", book1.status.label)
print("Copied Book:", book2.status.label)

let member1 = Member(name: "Anu", borrowedBooks: ["Swift Programming"])
print("Member Name:", member1.name)

let member2 = member1
member2.name = "Chitra"

print("First Member:", member1.name)
print("Second Member:", member2.name)
```

OUTPUT:

Output

```
Book Status: Available
Original Book: Available
Copied Book: Borrowed by Ben, due in 7 days
Member Name: Anu
First Member: Chitra
Second Member: Chitra
```

Task 2: Practice Task, Shapes and areas

Code:

```
enum Shape {
    case circle(radius: Double)
    case rectangle(width: Double, height: Double)
    case triangle(base: Double, height: Double)

    var area: Double {
        switch self {
        case .circle(let radius):
            return 3.14 * radius * radius
        case .rectangle(let width, let height):
            return width * height
        case .triangle(let base, let height):
            return 0.5 * base * height
        }
    }
}

struct Rectangle {
    var width: Double
    var height: Double

    var perimeter: Double {
        return 2 * (width + height)
    }

    mutating func scale(by factor: Double) {
        width = width * factor
        height = height * factor
```

```
    }}
let circle = Shape.circle(radius: 5)
let rectangle = Shape.rectangle(width: 4, height: 6)
let triangle = Shape.triangle(base: 8, height: 5)

print("Circle Area:", circle.area)
print("Rectangle Area:", rectangle.area)
print("Triangle Area:", triangle.area)



var rect1 = Rectangle(width: 10, height: 5)
print("Original Perimeter:", rect1.perimeter)

var rect2 = rect1
rect2.scale(by: 2)

print("Original Rectangle:", rect1.width, rect1.height)
print("Scaled Rectangle:", rect2.width, rect2.height)
print("Scaled Perimeter:", rect2.perimeter)
```

OUTPUT:

Output

```
Circle Area: 78.5
Rectangle Area: 24.0
Triangle Area: 20.0
Original Perimeter: 30.0
Original Rectangle: 10.0 5.0
Scaled Rectangle: 20.0 10.0
Scaled Perimeter: 60.0
```

Task 3: Challenge Task

```
struct Counter {
    var value = 0
    mutating func increment() { value += 1 }
}
class Box {
    var value = 0
    func increment() { value += 1 }}
var c = Counter()
c.increment()
print("Counter value:", c.value)
let b = Box()
b.increment()
print("Box value:", b.value)
var c2 = c
c2.increment()
print("c.value:", c.value)
print("c2.value:", c2.value)
let b2 = b
b2.increment()
print("b.value:", b.value)
print(b === b2)
```

OUTPUT:

Output

```
Counter value: 1
Box value: 1
c.value: 1
c2.value: 2
b.value: 2
true
```